

Runtime Validation Schema Engine

In mission-critical software environments, ensuring that internal functions receive parameters of the correct type is paramount for system stability. While Python's type hints provide static analysis, a runtime validation engine is required to enforce these constraints during active execution, preventing silent data corruption and elusive runtime bugs.

Implement a decorator factory `validate_schema` that provides robust runtime type enforcement for keyword arguments. The engine must cross-reference incoming `kwargs` against a predefined type schema, raising explicit `TypeError` exceptions if mandatory arguments are absent or if their data types do not align with the system's specification.

Example 1 Input:

```
VALIDATE {"id": "int", "name": "str"} {"id": 1, "name": "Alice"}
```

Output:

```
SUCCESS
```

Explanation: The provided keyword arguments (`id` as an integer and `name` as a string) align perfectly with the validation schema. The engine allows the function to execute successfully.

Example 2 Input:

```
VALIDATE {"id": "int"} {"id": "101"}
```

Output:

```
ERROR: Argument id must be <class 'int'>, got <class 'str'>
```

Explanation: The engine detects a type mismatch: the schema requires an integer for `id`, but a string was provided. The decorator blocks execution and raises a descriptive `TypeError`.

Function Parameter:

- `schema (dict)`: A mapping of parameter names to their required Python types.

- `kwargs`: The arguments being evaluated at runtime.

Returns

- Any: The original function's result on successful validation.

Constraints

- Use `functools.wraps` to ensure that function metadata is preserved.
- Raise `TypeError("Missing argument: <name>")` if an expected keyword is not provided.
- The type mismatch error must format the types using their string representation (e.g., `<class 'int'>`).

Input Format for Custom Testing Input from stdin contains: `VALIDATE <schema_json> <args_json>`.

Sample Case 0 Sample Input

```
VALIDATE {"price": "float"} {"price": 19.99}
```

Sample Output

```
SUCCESS
```

Explanation The floating-point price value is validated against the schema and passes.